

UNIX

M. I. M. P

Page No. 2711212
Date 17/11/24

Q.1 Explain Features of UNIX

Ans:

1. multi-user capability:

→ UNIX provide multi-user capability.

→ It allows user to use the resource of main computer.

→ In other words, at a time same computer resource such as hard disk, memory etc.

a) Dumb terminal: The terminal consist of only a keyboard and monitor known as dumb terminal.

b) Terminal Emulation: The terminal consist of keyboard and monitor with its own processor, memory and hard disk.

2) multi-tasking capability:

→ This feature allow user to carry out more than one job at the same time.

→ only one job run in the foreground while the other job run in the background.

→ we can switch jobs between background and foreground, suspend or terminate them.

3) Inter-process Communication:

→ This feature allow process to communicate with another user.

→ It allow putting data, exchange email or program to another user within the network.

4 Security:

→ UNIX provide three level security:

A) System Level Security:

→ In UNIX every user allocated login id and password.

B) Directory level Security:

→ In UNIX everything treated as file even directory or device are also consider as file. Read/Write

C) File Level Security:

→ In this there is file encryption.

→ ~~Security~~

5 Portability:

→ UNIX is written in high level language.

→ It makes it too easy to understand, change and move to other machine.

6 Open System:

→ It has an open architecture.

→ modification of the system is very easy because the source code is always available and free to download no license fees required.

7. window system: Window System

→ UNIX has pretty weak interface if it is command driven.

Later UNIX came up with X11 version.

→ It replace the command line with graphical objects in the form of menu, dialog box, icons or buttons.

8. System Call and Libraries:

→ UNIX is written in C, Fortran and C.

→ There are many commands available in UNIX that handle special functions called as system calls.

→ These calls are built into the kernel used to communicate with kernel directory.

9. Programming facility:

→ UNIX is highly programmable.

→ It provides control structures, loop and concept of variable.

→ It provides shell script and AWK programming.

10. Networking:

→ It allows user at one location to log into the system at the other end and enables user to access the resource of host machine.

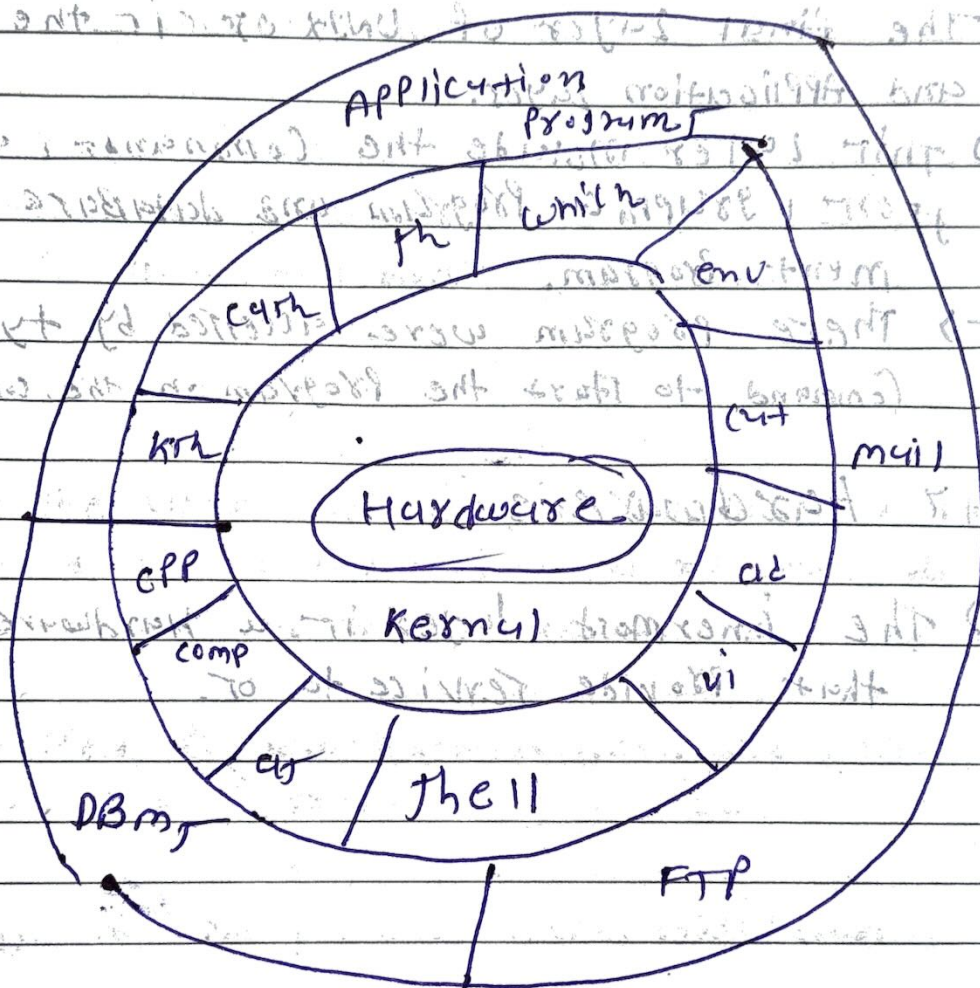
Q.2 Explain Architecture of UNIX? Ans

1. Kernel:
- The kernel of UNIX is the hub of the UNIX system.
 - Kernel is a set of routines mostly written in C language.
 - User programs that need to access the hardware use the services of the kernel.
 - The kernel is the heart of the UNIX system.
 - It handles the process, memory, file and device and network management for the operating system.

A Kernel is Program that:

- control all computer operations,
- coordinate all executing utilities,
- manage file, memory and handle interprocess scheduling of various program.

2. Shell:



- The shell is the program that sits between the user and the kernel.
- It is the interpreter that translate the commands that are typed into the terminal region.
- user can type commands directly into terminal, or they can create a text file containing a series of commands that can be sent to the shell.
- There are multiple shell that are used by the user.
- they include Bourne shell (sh), the C shell (csh), Korn shell (ksh) and Bourne Again shell (bash).

3) Utiltier and Application

- The final layer of UNIX OS is the Utiltier and Application Layer.
- This layer include the Communar, word pro-
fesson, graphic Program and database manage-
ment Program.
- There program were accessed by typing the
Command to start the Program in the Command line.

4) Hardware

- The innermost layer is a Hardware layer
that provide service to OS.

Q. 3 Explain Architecture of kernel?

Ans:

→ The Unix OS has three levels:

1. User level
2. Kernel level
3. Hardware level

1. User level:

→ The user level includes all the user programs, library and system call interface using which the user interacts with the kernel.

→ User program interacts with the kernel through a set of standard system calls.

System calls

fork()

It creates a new process

exec()

It executes a new program in process

wait()

To wait until a created process completes

exit()

To exit from a process execution

signal()

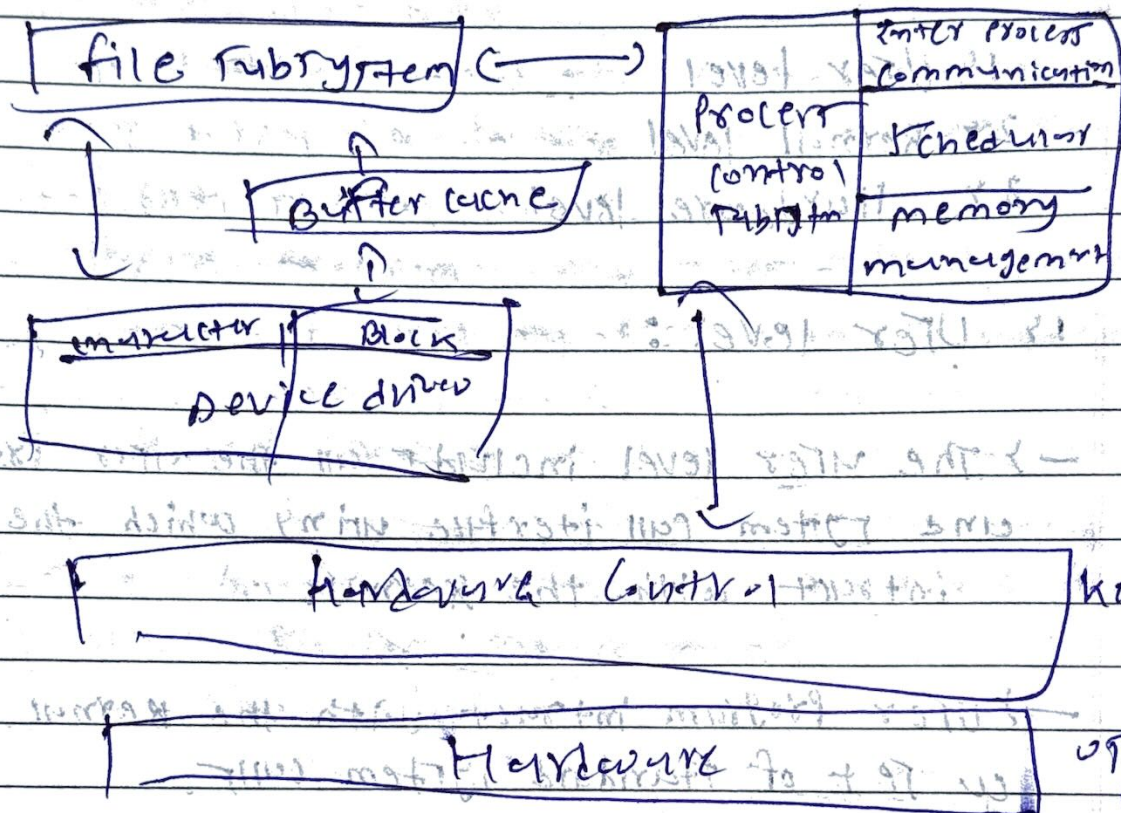
It controls the process response to external

brk()

It increases/decreases the data area of process

Library

System Call interface -



2.7 kernel level: The kernel has two major sub system:

→ The file sub system

→ It uses device drivers to read or write data to or from the disk files.

→ This is done using high speed cache memory which regulates the data flow between the kernel and the secondary storage device.

1) Character Device drivers:
 → character device drivers are used to interact with character device like terminal.

2) Block Device drivers: two types (→

→ Block device driver require high speed memory to transfer data to or from the kernel.

3) Process control subsystem

→ The process control subsystem and the file system interact with each other whenever a process need an executable file to be loaded into the memory for execution.

→ The scheduler module allocate the CPU to processes.

→ This scheduling of the CPU is done in turn,

where each process is given a time quantum.

→ inter process communication is implemented using advanced feature of piping between processes.

4) Hardware level

→ The hardware control consists of module for handling interrupt cycle of different device like disk, tape or terminal.

Q. 4 Explain features of shell?

Ans: 1. wildcard substitution in file names:

→ Carry out command on a group of file by specifying a pattern to match, rather than an actual file name.

2. Background processing:

→ Get up lengthy tasks to run in the background, freeing the terminal for concurrent interactive processing.

3. Command aliasing:

→ Give an alias name to a command or phrase.

→ When the shell encounters an alias on the command line or in a shell script.

4. Command history:

→ Record the commands you enter in a history file.

→ You can use this file to easily accept, modify, and reissue any typed command.

5. File name substitution : $\rightarrow$ $\therefore$ $\therefore$

$\rightarrow$ Automatically produces a list of file names on a command line using pattern-matching characters.

6. Input and output redirection :

$\rightarrow$ Redirects input away from the keyboard and redirect output to a file or device other than the terminal.

7. Piping :

$\rightarrow$ Link any number of commands together to form a complex program.

8. Shell Variable Initialization :

$\rightarrow$ Store data in user-defined variables and predefined shell variables.

Q. 5 Explain Z-node?

Ans:

→ we know that all entities in Unix are treated as files.

→ Z-node block stores information about the file and the location of data block where the content of file is actually stored.

→ The information related to all these files is stored in a inode table on the disk.

→ Each entry is made up of 64 bytes and contains the relevant details for that file.

File type: regular, directory, device, etc.

Number linker i.e. the number of aliases

Number numeric UID of the owner

Number numeric GID of the owner

File size of the file

Date and time of last modification of file

Table of contents: disk address of data blocks containing the file data

→ Z-node is accessed by a number called Z-node Number.

→ This number is unique for every file in the system.

→ The kernel always works with the memory copy but it periodically updates the disk copy with the content of the memory copy.

Q.6 Explain File System Structure?
ANS:

→ File system is one of the important pillar of UNIX system.

→ UNIX treats everything as a file.

The FILE :

→ The file is a container for storing information.

→ Unlike old DOS file, UNIX file does not contain a lot of character.

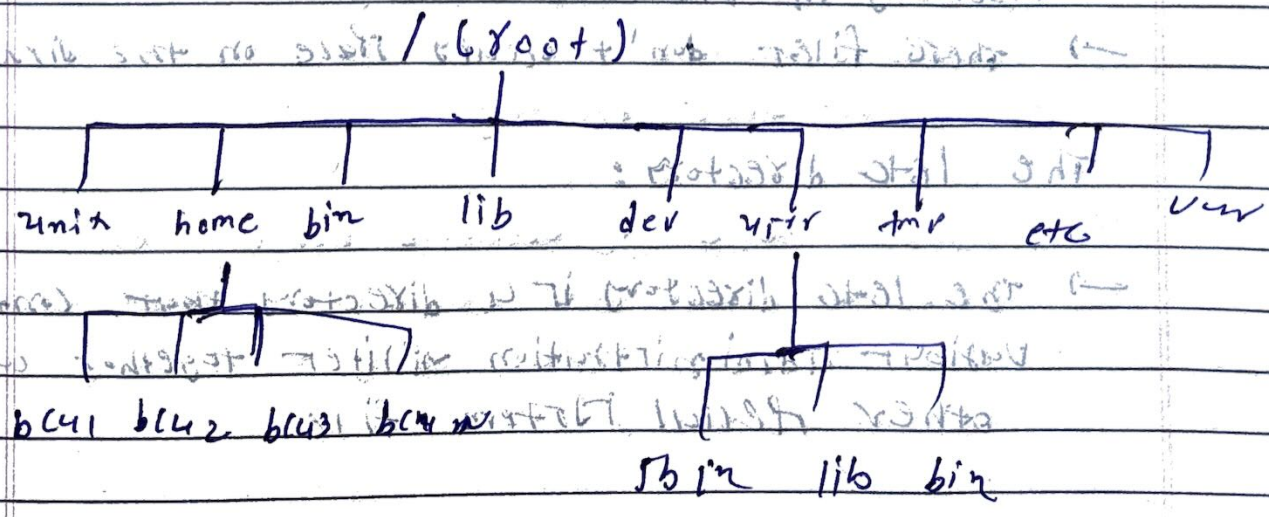
→ It contains only the information stored by the user.

→ Even the physical device like hard disk, memory, CD-Rom, printer, modem etc.

→ All data in UNIX is organized into files.

→ All files are organized into directories.

→ All other files are "derivative" of root.



The root directory :

- The slash / character alone denote the root of the file system tree
- It is also known as super user's home directory
- The root directory has no parent directory.

The /bin directory :

- The /bin directory contains commonly used unix commands that all users will have to use at some time
- The word "bin" stands for binary
- Binary files are executable files

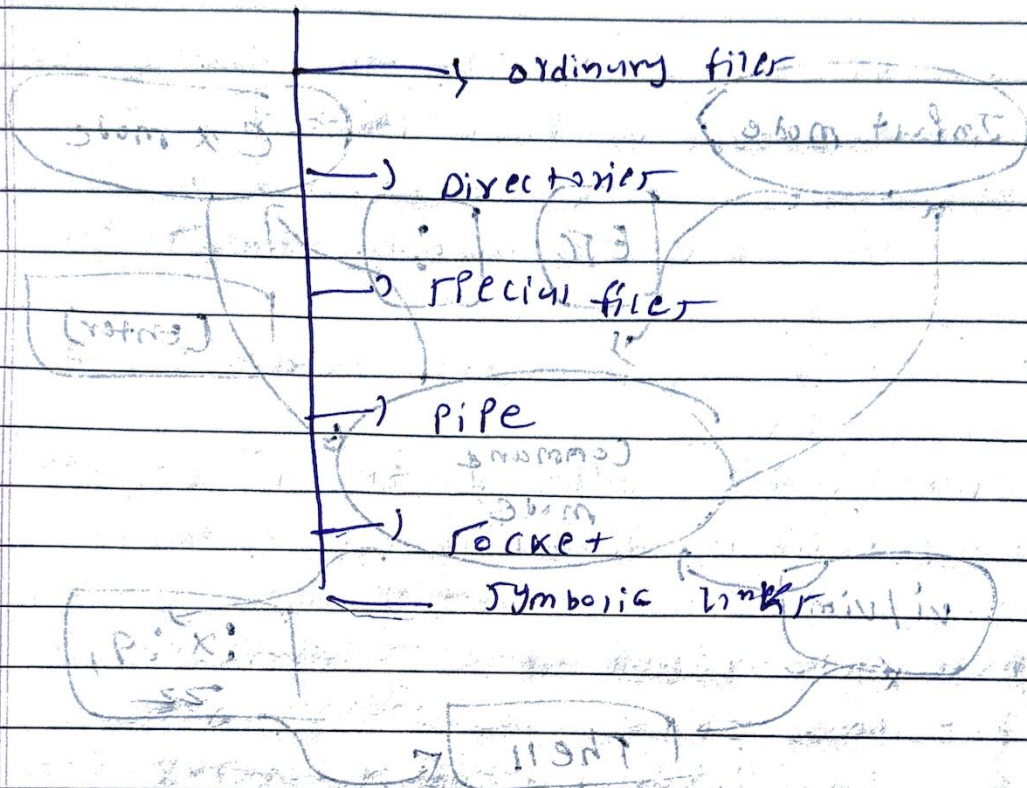
The /dev directory :

- All device files are stored in /dev directory or in its subdirectories.
- Unix treats devices such as printer & disk-storages as "device", terminal and even areas of the computer memory as file.
- These files don't occupy space on the disk

The /etc directory :

- The /etc directory is a directory that contains various configuration files together with other special system files

Types of UNIX files: 8.0
: 74A



next to it, the first part of the name is the file name and the rest is the extension.

For example, if the file name is 'file.txt', the extension is '.txt'.

It is important to know the difference between a file and a directory.

A file is a single unit of data, while a directory is a collection of files and other directories.

For example, if you have a directory named 'home', it contains files like 'desktop', 'documents', and 'downloads'.

Understanding the difference between files and directories is essential for managing your system.

One of the most common mistakes is to create a directory with the same name as a file.

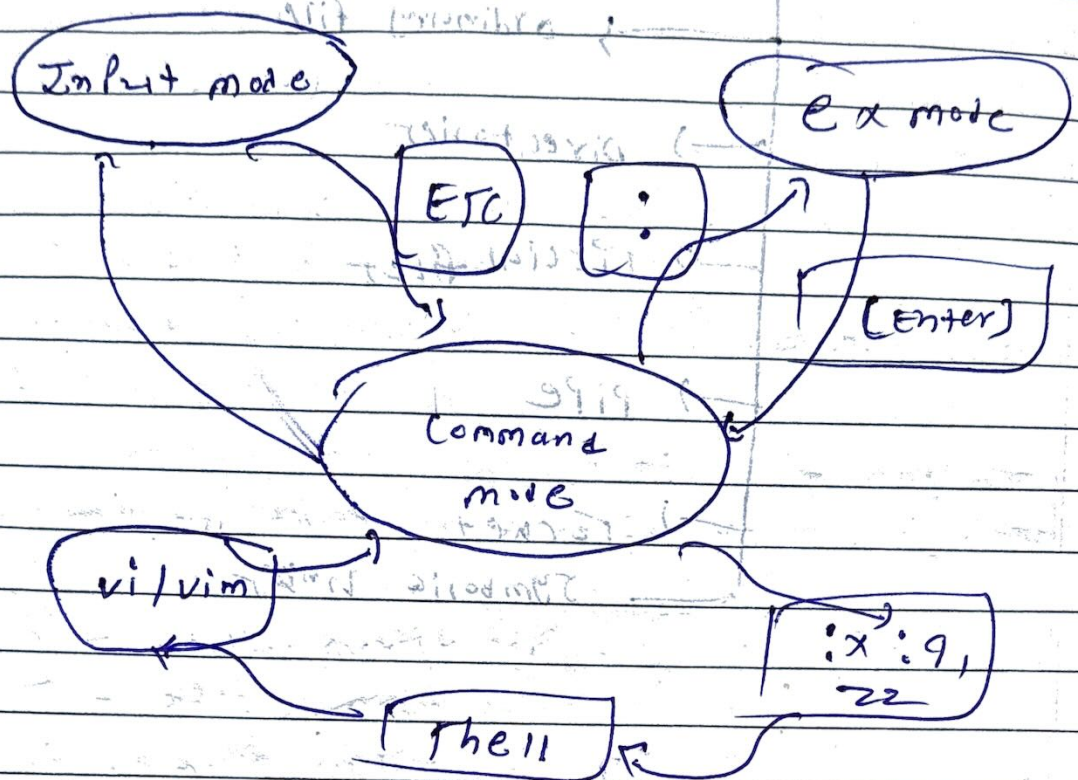
For example, if you have a file named 'home' and you create a directory named 'home', you will get an error.

To avoid this, you should use different names for files and directories.

For example, you can use 'home_dir' for the directory and 'home_file' for the file.

Q. 8 Explain VZ editor?

ANS:



→ vi editor is the most popular and classic text editor in the Linux family.

- 1> It is available in almost all Linux Distribution
- 2> It works the same across different platforms and distribution
- 3> It is user-friendly

→ Nowadays, there are advanced versions of the vi editor available, and the most popular one is VIM which is vi improved.

→ To work on vi editor

→ Command mode

→ Insert mode & vi editing commands

→ moving within a file

→ Saving and clearing the file

★ Command mode

→ The vi editor open in this mode, and it only understand commands

→ In this mode, you can move the cursor and cut, copy, paste the text

2. Insert mode

→ This mode is for inserting text in the file

→ You can switch to the insert mode from the command mode by pressing 'i'.

★ Starting the vi editor

To launch the vi editor - open the terminal and type

vi {filename - NEW} or {filename - EXISTING}

VZ Editing Commands

- i - insert
- a - write after cursor
- A - write at the end of line
- ESC - Terminate insert mode
- z - undo last change
- U - Undo all change to the entire line
- dd - delete line

Moving within a file

- k - move cursor up
- J - move cursor down
- h - move cursor left
- l - move cursor right

Saving and closing file

- Shift + ZZ - Save the file and quit
- :w - Save the file but keep it open
- :q - quit without saving

Q. 9 Explain Conditional Execution
Ans:

- The test command is used to check file types and compare values.
- you can also use it if used for
 - File attribute comparison
 - Perform string comparison
 - Arithmetic comparison

Syntax: [condition]

[condition]

or

[! condition]

or

[condition] && true-command

or

[condition] && true-command || false-command

Example

[5 == 5] && echo "yes" || echo "no"

[5 == 15] && echo "yes" || echo "no"

[5 != 10] && echo "yes" || echo "no"

Practical Shell Script

initializing two variable

a = 10
b = 20

check wheather

if [\$a == \$b]
then

echo "a is equal to b"

fi

check wheather they are not equal

if [\$a != \$b]

then

echo "a is not equal to b"

fi

Running

bash -f main.sh

Message - Chennai, Tamil Nadu, India
a = 10 b = 20
[a != b]
[a != b]

Q.10 Explain : grep

grep Utility :

→ The grep filter is used to search a file for a particular pattern of character, and display all records that contain the pattern.

→ The pattern is searched in the file is referred to as the regular expression.

→ grep stands for globally search a regular expression and print it.

grep [option] pattern [filename(s)]

→ In above syntax square bracket indicate option part.

→ The final filename is optional in the grep command.

→ without a filename grep expects standard input.

→ grep requires a pattern to represent the pattern to be searched for, followed by one or more filenames.

* Escaping Regular Expression: `\`

→ An expression formed with some special and ordinary characters, which is expanded by a command to match more than one string.

→ A Regular expression is always quoted to prevent its interpretation by the shell.

→ `grep` is representative UNIX command that silently return the prompt in case the pattern cannot be

* `grep` options:

→ `-c` (count)

`-n` (number)

`-v` (inverse)

`-h` (hide)

`-i` (ignore)

* `EGREP` Command: `egrep`

→ `Egrep` extend `grep` pattern matching capability.

→ It was invented by Alfred Aho.

→ It offer all the options of `grep`, but it must use `-f` option for one pattern for search.

* Storing pattern in file

→ If there are number of patterns, you have to mention `egrep` over the `-f` option to take each pattern from the file.

* **EGREP Command:**

→ `Egrep` accepts multiple patterns both from the command line and a file, but unlike `grep` and `egrep` does not accept regular expression.

→ So if the pattern to search for is single [string] or group of them, `grep` is connected.

Q.11 Explain sed utility and its various options.

Sed utility: It is a stream editor. It is used to edit text files. It is a command-line utility.

→ The term sed stands for stream editor and was derived by Lee McMahon.

→ Stream editor i.e., sed is derived from ed, a known single line editor.

→ Everything in sed is an instruction.

Syntax:

[sed [options] [instruction] [filename]]

→ An instruction consists of two components an address and a command which are enclosed within square brackets.

* Address:

→ An Address may be a line number, pattern or combination of them.

Two types of Address:

1. Line Number

2. A Pattern

→ Line address can be defn single line, or set of line or range of line

Address	Meaning
5 command	It applies command on 5 th line
8 command	" " " 14 th " "
15 command	It applies command on 15 th line

→ When you use pattern or regular expression to select lines, then it must be enclosed in brackets from the start to the end

Pattern / command	It applies command on line that
Pattern / command	It applies command on line which begin
Pattern / command	It applies command on line which end

* Command can be used in several ways

→ The sed supports several commands,

→ Command are used to apply action on specific line or lines.

1) Print

2) Line Number

4) modify

5) delete

6) substitute

7) hold space

8) hold space

Q.12) AWK utility

Ans:

→ AWK name come from the initial letter of three author: Alfred V. Aho, Peter J. Weinberger and W. Kernighan.

→ The AWK is powerful programming language designed as utility.

→ It reads the input file line by line and perform an action on a part of the line or on the entire line.

→ It also accepts regular expression for pattern matching which is language like programming structure, variable and several built in functions.

★ Some of the features of AWK utility

→ It has an ability to view a text file in term of record and field.

→ It use common programming constructs such as loop, condition and iteration.

→ It has ability to generate formatted report.

→ It allow the user to specify instruction a script.

Structure of awk script

Syntax:

awk option 'instruction' Filename (r)

An instruction part of awk program has three section, (i) BEGIN section, (ii) END

section, (iii) BEGIN section

selection criteria section
'END' section of [filename (r)]

→ awk counts each record of each input file.

→ for each record only selection criteria or an action or combination of both is available.

→ The pattern should always be entered with

→ Normally, awk take newline as a record separator and blank or tab as field separator

→ \$1 refers to the first field, \$2 refers to the second field and so forth of the current record while \$0 refers to

the entire current input record

Q. 13 Explain splitting, sorting, deleting, merging file ?

Ans

1. Splitting file:

→ At the name `split` command is used to split or break a file into the pieces of equal size in linux and unix system.

→ whenever we split a large file with `split` command then split output file default size is 1000 lines and its default prefix can be 'x'.

→ For its command for splitting

Header: `split -l 1000 file`

2. Sorting file:

→ `sort` command is used to sort a file, arranging the records in a particular order.

→ Various options for `sort` command

• `sort` command sorts the content of a text file, arranging lines by lines.

→ It supports sorting alphabetically, it reverse order by `-r` option can also remove duplicate

1. Sort Command : It is used to sort the files.

- o option : output file
- r option : reverse order
- n option : numerically sorted
- nr option : numerical data in reverse order
- C option : file names are sorted on the basis of character.

3. Comparing file :

→ cmp command in linux is used to compare two files byte by byte and help you to find out whether the two files are identical or not.

→ when cmp is used for comparing between two files it reports the location of the first mismatch to the screen if different or found and if not different it found.

Syntax :

cmp [option] file1 file2 [r] [p] [q]
 [r] : report the location of the first mismatch to the screen
 [p] : print the location of the first mismatch to the screen
 [q] : quit after the first mismatch

Example :
 \$ cmp file1 file2
 file1 file2 differ at line 10 column 5

Q. 14 Explain write, wtt, merge, wll, smail.

Ans write

→ The write command allows user to communicate with other user.

→ It is an on-line communication command.

→ It sends a message to another user who is currently connected to the unix system.

Syntax :

write user [username]

→ The write command provides a facility to communicate with other user, by copying lines from his terminal to other user terminal.

→ When user runs write command, the other user on the other hand gets a message on his terminal in the following form:

message from username@host on daythly

\$ write bcr10 & enter

- (i) sender terminal is write protected and try send
- (ii) Receiver is not logged in
- (iii) Receiver terminal is write protected

C31 merge :

→ It control write access to your terminal by other.

→ It will allow or disallow other user to write to your terminal.

Syntax :

`merge [y/n]`

→ without argument it display the current state your terminal

`$ merge`

if [y/n]

→ It has two option which determine the write access to your terminal.

→ use of merge command

option	meaning
y or Y	It allow write access to your terminal
n or N	It disallow write access to your terminal

→ If a user wish to disallow write access to his terminal then he typed this command

`$ merge n`

wall () :

- It sends a message on the terminal of all users who are currently logged in to the UNIX system.
- This command is specially designed for the super user.
- The wall command enables the super user to write to all interactive of the users giving write permission to their terminal.

Syntax :

wall [message]

- The message can be given as argument

\$wall "Good morning nites"

good morning nites

- The message can be sent as the standard input from a terminal.

mail: subject & body text & address of sender

mail: off-line communication command

It send and receive mail, unlike write and talk allow user to send mail to user even if they are not logged in to the machine.

Syntax: mail [options] [address]

mail [-s subject] [-c cc-address] [-b bcc-user]

[-r recipient] [-f file] [-i name] [-m message]

mail [-u user] [-s subject] [-c cc-address] [-b bcc-user] [-r recipient] [-f file] [-i name] [-m message]

When user receives a mail message the reverse things

view of mail on the terminal
have in a mailbox
save it in file
Reply to it

Command	Action
+	It display next message
-	It display previous message
n	It display only header of all
b	It delete only header
d n	It delete message n
u n	It undelete message n

2.16 what is the types of shell

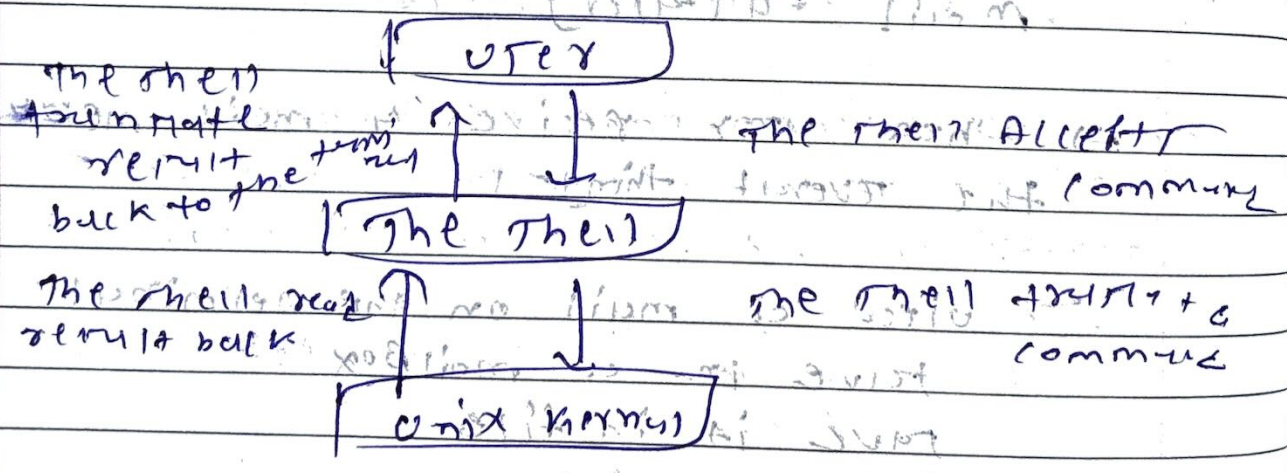
ANS: → shell is nothing but command interpreter

→ This is most widely used utility program on all Unix systems

→ shell is the key to interact with the operating system

→ The shell is the ~~key to interact~~ ^{first program} that is written in 'C'

→ The shell is responsible for returning the result of the computer or terminal



Types of shell

1. Bourne shell

2. C shell

3. Korn shell

1. Bourne Shell

→ This is one of the most widely used shells in Unix world.

→ It is developed by Steve Bourne at AT&T Bell Laboratory in late 1970.

→ It is primary Unix command interpreter and comes along with every Unix system.

→ It provides dollar prompt '\$'.

→ The executable file name is 'sh'.

→ Mr. Bill Joy developed it at University of California.

→ It has two advantages over Bourne shell.

→ It allows taking of command history feature means it stores previously used commands in a file.

→ It provides % prompt as a trademark of C shell.

→ The executable file name is 'csh'.

3. Korn Shell

→ It is very powerful and is an improvement of Bourne shell.

→ The Korn shell includes all the enhancements which are in the C shell like command history, etc.

→ It is developed by David Korn at AT&T Bell.

→ It provides dollar prompt '\$'.

→ The executable file name is 'ksh'.

Q. 16 Explain Booting sequence processing in detail.

Ans:

→ 1) Rom diagnostic circuitry.

→ hardware and memory test are performed.

2) Boot loader is loaded.

→ The Boot loader is loaded into Ram by the Rom.

3) Kernel is loaded.

→ The kernel program is called by the user.

→ Recall the user need to enter the name of the program.

4) kernel initialization timer placed.

→ It involves performing memory tests, initializing

the device through device driver, r/wapper, the kernel process and the init process.

5) The init or the init program is

invoked by the kernel.

6) The init program

→ This process has the PID 1.

7) The getty process

→ The init program also invokes the

getty program which establishes a communication with the terminals or Unix users.

8) The user can log in to the system.

84 login Program 1

→ The user types login Name and Password, getty transfer control to a login program to verify the login-name and password enter by user.

init → getty → login → shell